

BANKING DATABASE DESIGN

Schema, Indexing, and Query Performance

Mid-Semester Progress Report

Written By:

Domenic Telles

domenict0518@arizona.edu

Alexander Gomez

alexandergomez01@arizona.edu

Sandra Castrejon

sandraelenac@arizona.edu



**University of Arizona – College of Engineering
ECE 501C – Fall 2025**



Table of Contents

Abstract.....	2
Introduction.....	2
Motivation	2
Related Works	2
Background.....	4
Conceptual, Relation Modeling & Schema Refinement.....	4
Relational Algebra & Query Foundation.....	4
Normalization and Decomposition	4
Indexing and Query Optimization	4
SQL Standards.....	5
Tools & Technologies	5
Python	5
SQLite3	6
Banking Database Simulation Set Up.....	6
Project Scope	6
Database Schema.....	6
Data Generating & Loading DB.....	8
Banking DB Query Experiments	9
Experiment 1: Baseline.....	9
Experiment 2: Optimization	9
Experiment 3: Upscaling Sample Sizes Data	10
Upscaling Sample Size Data by a Factor of 3.....	10
Upscaling Sample Size Data by a Factor of 5.....	10
Upscaling Sample Size Data by a Factor of 8.....	10
Upscaling Sample Size Data by a Factor of 10.....	10
Experiments Results Analysis... So Far	11
Lessons Learned.....	12
Resource Software Application	12
Milestone Achievements.....	13
Plan To Move Forward	13
References.....	13



Abstract

This project implements a banking database (DB) system designed to evaluate the impact of schema normalization, indexing, and query design on data performance and functionality. The DB captures banking entities, including customers, accounts, cards, loans, and transactions, and demonstrates how relational design supports financial operations such as balance management, statement generation, and anomaly detection.

Introduction

Modern financial institutions rely heavily on well-designed DB to store, retrieve, and analyze customer and transaction data. In academic settings, however, real banking DBs are often restricted due to privacy laws such as the General Data Protection Regulation (GDPR) [1] and the Gramm-Leach-Bliley Act (GLBA) [2]. Both of these laws impose strict requirements on how personally identifiable financial information is collected, processed, and shared, making it nearly impossible to access real banking data for research or instructional purposes. As a result, many public datasets used in education lack realistic temporal and monetary patterns, limiting their effectiveness in studying real-world data management (DM) problems.

This project aims to bridge that gap by offering a reproducible, synthetic banking workload that can be used to teach and evaluate DM principles such as schema normalization, indexing, query optimization, and performance analysis. The project focuses on two primary goals. The first step is to design a normalized relational schema that enforces data integrity. The second is to measure how a DB design and indexing affect query latency and scalability under transactional patterns. By simulating real banking behavior, this project enables hands-on experimentation with DB performance and optimization concepts.

Motivation

Modern banking systems process massive volumes of transactional data daily, whether it is credit card transactions, loan repayments, or paying off credit card debts; DBs are essential in today's modern banking systems. Requiring DBs that are not only functionally correct but also highly optimized for performance, scalability, and reliability. Each financial transaction, such as deposits, withdrawals, transfers, and loan payments, must be accurately recorded and efficiently retrieved. These requirements make banking an ideal domain for studying relational DB design and optimization.

The motivation behind this project is to bridge theoretical DB principles with real-world DM challenges by designing and analyzing a simplified yet realistic banking database system. While many academic exercises focus on static queries or small-scale datasets, this project simulates the dynamic financial operations by generating synthetic yet structured data and evaluating how DB design decisions, such as normalization, indexing, and query planning, affect performance. Ultimately, this project aims to provide a practical learning experience that reinforces theoretical understanding through hands-on experimentation. Indexing baseline and indexed query performance, the goal is to understand and observe how schema design, indexing strategy, and query performance contribute to achieving measurable improvements in system responsiveness.

Related Works

Several public datasets and benchmarks aim to capture transaction-like behavior, such as the Transaction Processing Performance Council [3], which focuses on two specific benchmarks: TPC-C (transaction processing) and TPC-H (decision support). TPC-C is an industry-standard benchmark for online transaction processing that models a wholesale supplier with a mix of five concurrent transaction



types (new order, payment, and delivery) over a nine-table schema. Results are reported as transactions per minute to reflect sustained throughput under load. TPC-H is a decision-support benchmark comprising business-oriented queries and concurrent data modifications, designed to scan large volumes of data and answer complex analytical questions. However, TPC-C and TPC-H are designed for large enterprise benchmarking and are not easily adapted for teaching or lightweight experiments. This project aims to explicitly preserve entity relationships (Customer–Account–Transaction) and temporal dependencies. While also including reproducibility controls and tunable parameters for scale. At the same time, it automates an experiment loop (baseline vs. optimized) to measure query performance metrics within this environment.

Garcia-Molina, Ullman & Widom's Database Systems: The Complete Book [4] provides the conceptual and practical foundation for how the Banking DB system is structured, managed, and optimized. *The Complete Book* details the relational model and Standard Query Language (SQL), which directly support the project's schema design, where tables such as customers, accounts, and transactions are modeled as normalized relations with clearly defined keys and referential constraints [4]. These relationships mirror how real banks enforce data integrity between clients, accounts, and transaction histories. *The Complete Book's* explanation of query processing and optimization translates directly into how the project benchmarks SQL queries [4]. Concepts such as algebra-to-SQL rewrites and cost-based optimization explain why some query plans run faster than others, as well as comparisons between baseline and indexed performance runs. The book's emphasis on transaction management, concurrency control, and recovery introduces the ACID principles (Atomicity, Consistency, Isolation, and Durability). These principles are vital to banking operations: ensuring that a fund transfer either completes in full or not at all, that accounts remain balanced after each operation (consistency), and that concurrent users can safely access the system without data conflicts (isolation). While this project's implementation is simplified, its structure and schema constraints are modeled to reflect these same transactional guarantees.

Ramakrishnan & Gehrke's Database Management Systems [5] provides the foundational principles that directly inform the design and performance evaluation of banking DB. In a real-world banking context, millions of customer transactions must be efficiently stored, indexed, and queried for accurate and timely decision-making. *Ramakrishnan & Gehrke's* discussion of record and page organization guides how the DB physically stores tables, such as transactions and accounts, to minimize Input/Output (I/O) operations during high-volume queries, such as retrieving customers' monthly statements [5]. Their treatment of B+-trees and hash indexes directly relates to how this project applies indexing: B+-trees are used for range queries, similar to scanning all transactions for an account by date. In contrast, hash indexes support equality lookups (like finding a customer or account by ID) [5]. The DB joins customers, accounts, and transactions, a pattern typical in banking systems, and query optimizers use data statistics to determine efficient join sequences. By running baseline and indexed experiments and reporting 50th and 95th percentile latencies, this project puts *Ramakrishnan & Gehrke's* cost-estimation theory into practice, showing how physical design choices alter query plans and execution costs.

Together, *Garcia-Molina, Ullman & Widom's Database Systems: The Complete Book* [4] and *Ramakrishnan & Gehrke's Database Management Systems* [5] emphasize that practical experimentation with indexing, query planning, and cost-based optimization strongly reinforces the theoretical understanding of DB performance. Both bridge the gap between theory and implementation by demonstrating how relational design, physical indexing, and query execution strategies influence system efficiency and performance.



Background

The design and implementation of a DB system rely on several theoretical foundations that govern data organization, integrity, performance, and the use of technology tools. *Database Systems Concepts*, by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan [6], emphasizes that efficient database systems result from combining conceptual modeling, relational theory, normalization, and query optimization. Collectively, these theories shaped the design, implementation, and experimental validation of this project's banking DB system, ensuring that the project adhered to both academic rigor and practical performance standards.

Conceptual, Relation Modeling & Schema Refinement

The Entity-Relationship (ER) model provides the conceptual foundation for designing DB schemas [6]. It represents real-world entities, their attributes, and relationships in a structured format. In this project, entities were modeled using this framework, with one-to-many and many-to-many relationships formally represented through primary keys (PKs) and foreign keys (FKs). These relationships preserve referential integrity and enable the logical connection between customer data and financial activity. Relational Model, which translates ER constructs into formal table definitions characterized by tuples, attributes, and domains [6]. The project's schema design adheres to the formal rules of the relational model, ensuring that each table maintains atomic attributes, unique identifiers, and consistent data types. The Relational Model and ER model were essential for developing the Data Definition Language (DDL) scripts that establish structure and enforce relational integrity across entities. Integrity constraints such as PK, FK, and CHECK were enforced to ensure data consistency and validity across the schema, to ensure that all relationships remain logically sound during data manipulation.

Relational Algebra & Query Foundation

Relational algebra defines the formal operations used to manipulate data. Operators such as selection (σ), projection (π), join ($\bowtie$), and aggregation (γ) underpin the semantics of SQL queries used throughout the project. Understanding relational algebra allowed the team to optimize complex join and filtering operations in SQL by recognizing how multiple algebraic expressions translate into query plans. This theoretical basis informed the design of queries in experiments comparing baseline scans and optimized index-based retrievals.

Normalization and Decomposition

Normalization theory is a process used to eliminate redundancy and update anomalies [6]. The project employed Third Normal Form (3NF) and Boyce–Codd Normal Form (BCNF) as guiding principles in its schema refinement. Each entity was decomposed into smaller relations until all functional dependencies were satisfied by normalization rules. This ensured that changes to one table would not cause unintended inconsistencies elsewhere, thereby improving maintainability and query performance. Normalization also enabled efficient indexing and join operations in later optimization stages.

Indexing and Query Optimization

The performance layer of the system draws heavily from the principles that discuss storage, indexing, and query optimization [6]. The storage structures and the importance of minimizing I/O operations are directly reflected in this project's measurements of latency and index build time. The logarithmic search cost of a B+ tree is given by:



$$\text{Cost}_{\text{search}} = O(\log_F(N))$$

where F denotes the fan-out (average pointers per internal node) and N the number of indexed entries. This efficiency relationship explains why indexed lookups in the optimized schema significantly outperformed full table scans in the baseline configuration. Query processing and optimization, including cost estimation and the selection of an execution plan [6]. The theoretical total query cost is expressed as:

$$\text{Cost}(Q) = \text{I/O cost} + \text{CPU cost} + \text{communication cost}$$

This model mirrors the project’s experimental evaluation, where baseline queries, executed without indexes, incurred high I/O cost due to sequential scans, while optimized configurations reduced both I/O and CPU costs. The observed performance gains empirically validated this cost-based optimization framework.

SQL Standards

SQL is the standardized language used to define, manipulate, and control data in relational DB systems. The SQL standards were initially developed by the American National Standards Institute (ANSI) [7]. Moreover, it was later adopted by the International Organization for Standardization (ISO), providing a unified framework for database design and interoperability across different DB management systems (DBMS). The project’s schema design and query implementation strictly follow SQL-92 and SQL-99 (also referred to as SQL2 and SQL3) conventions.

1. DDL:

DDL statements define the structure and constraints of DB objects. In this project, DDL scripts were implemented following SQL-92 (SQL2) syntax, ensuring consistent table creation, PK and FK enforcement, and referential integrity.

2. Data Manipulation Language (DML):

DML operations such as *INSERT*, *UPDATE*, *DELETE*, and *SELECT* manage the stored data. In the experiments, DML queries were used to populate tables, simulate transactions, and perform analytical queries. SQL-99 (SQL3) extensions such as subqueries and aggregate functions (SUM, AVG, COUNT) were also employed to enhance expressiveness and reduce query complexity.

3. Data Control and Transaction Language (DCL/TCL):

The project incorporated transaction commands (*BEGIN TRANSACTION*, *COMMIT*, *ROLLBACK*) to represent the ACID compliance operations conceptually. While SQLite handles transaction isolation implicitly, the project’s SQL structure adheres to the SQL-99 (SQL3) model for consistent updates, particularly in multi-step banking operations.

Tools & Technologies

Several tools were used to design, implement, and evaluate the banking DB system. The primary environment integrates Python [8] and SQLite3 [9] for executing, optimizing, and benchmarking queries. This section outlines the purpose and role of each tool in the project.

Python

Python is the primary programming language for data preprocessing, automation, and experimental evaluation. Its extensive standard library, particularly *NumPy*, *Pandas*, and *Matplotlib*, enabled the maintenance of a consistent and modular workflow throughout the DB pipeline [8]. Python scripts were implemented to generate realistic banking data, including customer profiles, account balances, transactions, and loan records. The data was organized and exported as comma-separated values



(CSV) files, which SQLite then ingested. This provided complete control over data volume and schema consistency during the generation process.

SQLite3

SQLite3 was chosen as the DBMS due to its file-based architecture and full compliance with SQL standards as mentioned above. It provides a balance between simplicity and functionality, making it suitable for performance benchmarking in a controlled local environment without the overhead of a client-server setup [9]. The DB was constructed using DDL scripts to ensure consistency in schema deployment. The DDL scripts defined tables, PKs, FKs, and constraints enforcing data integrity across entities.

Banking Database Simulation Set Up

The Banking DB Simulation integrates schema design, synthetic data generation, query benchmarking, and optimization into a cohesive experimental platform by adhering to SQL standards, relational theory, and normalization principles as mentioned in the section above.

Project Scope

The Banking DB Simulation was developed to model and analyze core financial operations, including customers, accounts, transactions, loans, and cards, within a relational DB environment. The project's goal was to design a normalized and scalable schema, implement both baseline and optimized query configurations, and evaluate performance through controlled experiments measuring query latency, index build time, and data scalability. The simulation aims to replicate real-world DM challenges faced by financial institutions, such as ensuring referential integrity, maintaining transactional consistency, and supporting analytical queries for fraud detection, loan tracking, and account summaries. To maintain reproducibility, all experiments were implemented using SQLite3 as the DBMS and Python for data generation, query automation, and visualization.

Database Schema

The DB schema was designed in accordance with the principles of relational modeling and normalization mentioned above. Each entity represents a distinct operational unit within the banking domain. All relationships are defined through PKs and FKs to preserve referential integrity, as shown in Figure 1 below, and the ER Model for this project is shown in Figure 2.

Entity	Primary Key	Foreign Key	Other Attributes	Total Attributes
Account	account_id	branch_id account_type_id	account_number, is_active, balance, account_type	7
AccountCustomer	account_id customer_id	account_id customer_id	is_primary	3
AccountType	account_type_id		name, description, interest_rate	4
Address	address_id		line1, city, state, postal code	4
Branch	branch_id	address_id	branch_code, contact_info, routing_number	5
Card	card_id	account_id	last4, brand, status, activated_at	6
Customer	customer_id		name, phone, id_number	4
CustomerAddress	customer_id address_id address_type	customer_id address_id	is_primary	4
Loan	loan_id	customer_id	Principal, rate_apr, term_months, start_date, payment_day, loan_type	8
Transactions	txn_id	account_id	ts, amount, merchant, category, description	6

Figure 1: DB Schema

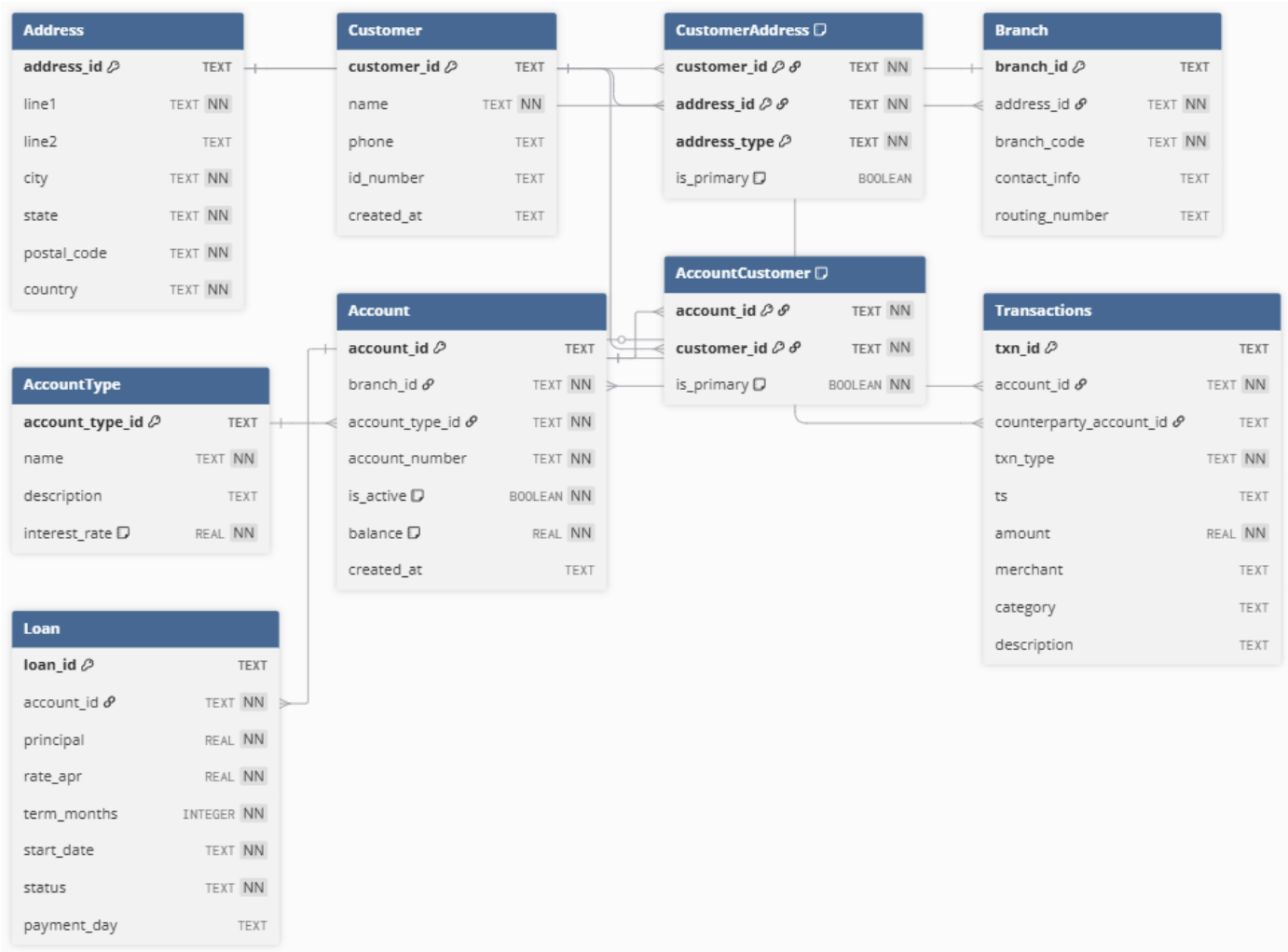


Figure 2: ER Model

Figure 3 shows an example of the DDL. This is the SQLite DDL used to build the Banking DB, defining tables, data types, PK, and FK constraints. Notably, the *Customer* junction table uses a composite PK (*customer_id*) to enforce the many-to-many relationship between customers and accounts.

```
-- Customer: PK customer_id
CREATE TABLE Customer (
  customer_id TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  phone       TEXT,
  id_number   TEXT
);
```

Figure 3: Schema DDL Example



Data Generating & Loading DB

A synthetic dataset was created using a Python script (*synth_data.py*), which generated CSV files representing each entity. The data generation used *NumPy* and *Pandas* to produce realistic and randomized records for: customers, accounts, transactions, loans, cards, and branches.

An Extract, Transform, and Load (ETL) procedure was implemented to automate the complete data ingestion process and ensure data reproducibility. The ETL operation was executed through a Python script, *load_sqlite.py*, which performed the following three stages:

1. Extract (E): Synthetic data were generated through the *synth_data.py* module and exported as structured CSV files for each entity (customers, accounts, loans, and transactions).
2. Transform (T): The Python scripts standardized data types, formatted timestamps, and verified schema consistency across all keys and foreign-key relationships, ensuring that data adhered to the normalization.
3. Load (L): Using *load_sqlite.py*, the transformed datasets were inserted into the SQLite DB, where constraints, composite indexes, and materialized views were applied.

The output of this ETL process is the Banking DB, titled ***FinalBankDB.sqlite***, which serves as the foundation for all subsequent experiments. By integrating ETL into the simulation pipeline, the project guarantees consistent data integrity, efficient data reloading for re-runs, and scalable testing of indexing and optimization features. The Banking DB Simulation flowchart in 4 illustrates this process. It depicts the entire data life cycle, from synthetic data generation and transformation to loading relational tables, followed by the query benchmarking stage. Figure 5 is also a screenshot representation of the actual *FinalBankDB.sqlite* DB, showing how each stage (ETL and Query) connects to produce the final experimental dataset used for baseline and optimized evaluations discussed in this report's next section.

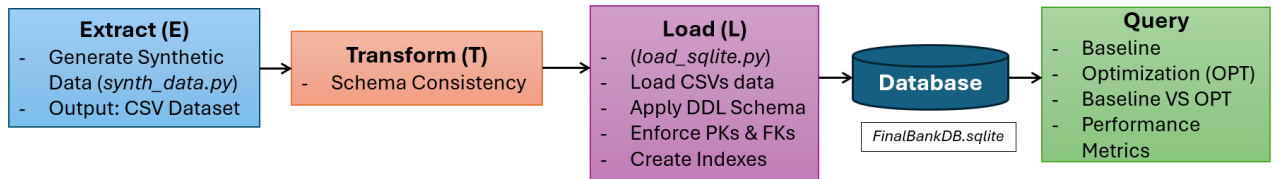


Figure 4: Banking DB Simulation Flowchart

account...	Filter	account_t...	account_n...	balance
1 A_NQETMNI3GO B004	1	checking	A_NQETMNI3GO	1203.3
2 A_XLSUTOTHLQ B005	1	checking	A_XLSUTOTHLQ	754.52
3 A_PPCADUVL91 B005	2	savings	A_PPCADUVL91	6189.66
4 A_D60UGLBB8N B005	1	checking	A_D60UGLBB8N	1184.94
5 A_UWB8N6EK6X6 B005	2	savings	A_UWB8N6EK6X6	2712.93
6 A_48FIY5PU5W B004	1	checking	A_48FIY5PU5W	773.65
7 A_3N4G2JAYB1 B003	1	checking	A_3N4G2JAYB1	560.89
8 A_TKUDE358TR B003	2	savings	A_TKUDE358TR	1104.21
9 A_9QDI3G13IW B003	3	loan	A_9QDI3G13IW	3561.06

Figure 5: Screenshot of the *FinalBankDB.sqlite*



Banking DB Query Experiments

This projects evaluate performance query (Q) using a fixed, labeled SQL workload (Q1–Q11) that reflects common access patterns: name lookups and fan-outs (Q1–Q3), point-in-time balance (Q4), recent-window activity (Q5–Q6), daily branch aggregates (Q7–Q8), top spenders (Q9), dispersion across branches (Q10), and loan exposure by branch (Q11). Each experiment runs the same workload while changing one factor at a time (indexes/settings or dataset size) to ensure fair, explainable comparisons. An *indexes_optimized.sql* script was created to accelerate this workload (Optimization), and a *benchmark.py* harness executes the named queries, averages runtimes for both Baseline and Optimized modes, and saves results to separate folders for comparison. Each experiment, **Baseline**, **Optimization**, and **Upscaling**, is evaluated with the fixed workload in *queries_workload.sql*:

- **Q1:** Lists customers whose names start with familiar initials and counts how many accounts each has.
- **Q2:** Same as Q1 but restricted to the top five initials by frequency in the dataset.
- **Q3:** Returns every customer with the number of accounts linked to them.
- **Q4:** Computes the ledger balance for one account by summing all its transactions up to now.
- **Q5:** Shows the most recent transactions in the last 30 days for a single demo account.
- **Q6:** Shows recent (30-day) transactions for the most active accounts, ordered newest first.
- **Q7:** Aggregates daily transaction counts and net amounts per branch over the last 30 days.
- **Q8:** Splits daily branch totals into credits and debits over the last 30 days.
- **Q9:** Ranks customers by spending (outflows) in the last 90 days and returns the top results.
- **Q10:** Counts how many distinct branches each customer used in the last 30 days and lists the top dispersers.
- **Q11:** Sums active loan counts and principal by the customer’s primary branch.
- **Q12:** TBD
- **Q13:** TBD
- **Q14:** TBD
- **Q15:** TBD

Experiment 1: Baseline

The objective of “Experiment 1: Baseline” is to measure the raw query latencies on the unindexed DB with default SQLite settings to establish a fair performance reference. This defines the “baseline” for all comparisons. The Baseline data generated consisted of 1,000 customers, 1,830 accounts, 60,000 transactions, 130 loans, 812 cards, and 5 branches.

Experiment 2: Optimization

The objective of “Experiment 2: Optimization” is to apply targeted indexes to convert broad scans into selective range scans, then re-measure the same workload to quantify speedups and any regressions. The goal is to demonstrate how design choices enhance latency and efficiency compared to the Baseline (Experiment 1).

The *queries_workload.sql* file defines a consistent set of labeled SQL queries (Q1–Q11) representing a realistic cross-section of banking operations, including customer lookups, transactional summaries, loan aggregation, and branch-level activity. This standardized workload ensures that all experiments, Baseline, Optimization, and Upscaling, are evaluated under identical query conditions. Each query has been designed to test a specific aspect of DB performance, such as indexing efficiency, join



complexity, and temporal filtering. The *benchmark.py* script uses this workload as input, automating query execution and collecting performance metrics for comparative analysis. Table 1 presents the results of Experiment 1 (Baseline) and Experiment 2 (Optimization) side by side.

Table 1: Experiment 1 & 2 Results

Query	Baseline	Optimization	Rows returned
	Average Time (seconds)	Average Time (seconds)	
Q1	0.002649	0.008352	546
Q2	0.002356	0.006423	546
Q3	0.003223	0.004687	1000
Q4	0.000024	0.000026	1
Q5	0.000055	0.000074	27
Q6	0.21548	0.037727	100
Q7	0.392043	0.074861	110
Q8	0.51137	0.08457	110
Q9	0.592367	0.053515	1000
Q10	0.529291	0.045118	1000
Q11	0.000471	0.000229	5

Experiment 3: Upscaling Sample Sizes Data

The objective of Experiment 3 is to evaluate the scalability and robustness of the Banking DB system by progressively increasing the dataset size. This upscaling experiment tests how query performance, index efficiency, and overall system responsiveness behave under larger workloads that mimic real-world banking growth. Using the synthetic data generator (*synth_data.py*), the dataset was scaled by factors of 3, 5, 8, and 10 to produce expanded versions of the customer, account, and transaction tables. Each scaled dataset was loaded into separate database instances to prevent interference with previous experiments. The same SQL workload (Q1–Q11) and benchmarking framework (*benchmark.py*) were applied across all scales, allowing for consistent measurement of query latency and index build time. ***This work is still ongoing and will be incorporated into the final report.***

Upscaling Sample Size Data by a Factor of 3

Table 2: Upscaling Data 3X Baseline VS Optimization

[\[Enter Table\]](#)

Upscaling Sample Size Data by a Factor of 5

Table 3: Upscaling Data 5X Baseline VS Optimization

[\[Enter Table\]](#)

Upscaling Sample Size Data by a Factor of 8

Table 4: Upscaling Data 8X Baseline VS Optimization

[\[Enter Table\]](#)

Upscaling Sample Size Data by a Factor of 10

Table 5: Upscaling Data 10X Baseline VS Optimization

[\[Enter Table\]](#)



The *benchmark.py* script serves as the core experimental engine of this project. Its primary purpose is to systematically measure and compare query performance across different configurations, specifically, the Baseline, Optimized, and Upscaled DB environments. By executing a fixed SQL workload (Q1–Q11) against the DB, *benchmark.py* records metrics such as average query latency, execution time variance, and scalability under increasing data volumes. This automation ensures that each experiment is performed under consistent conditions, allowing for reproducible and fair performance comparisons. The script not only validates the impact of indexing and schema optimization strategies but also quantifies their real-world performance benefits. Ultimately, *benchmark.py* supports the project’s objective: to demonstrate how DB design choices, particularly indexing and query optimization, directly influence efficiency, scalability, and overall system responsiveness.

Experiments Results Analysis... So Far

This section compares average latencies for Q1–Q11 under two conditions: the **Baseline** (Experiment 1) configuration and the **Optimization** (Experiment 2) configuration. The *benchmark.py* harness executed each query with one warm-up run followed by multiple timed runs, saving the averages and row counts for each mode to separate result folders to ensure a fair comparison. We quantify improvement using the standard speedup metric, defined as:

$$\text{Speedup}(x) = \frac{\text{Baseline Avg. Time}}{\text{Optimized Avg. Time}}$$

The equation above was used to calculate the Speedup(x) shown in Table 6. Values greater than 1 indicate that the optimized run is faster. In contrast, values less than 1 indicate a regression.

Table 6 reports the per-query averages, row counts, and computed speedups. The time-window and grouping queries (Q6–Q10) benefit the most, improving by approximately 5× to 12×, because the optimizer can exploit the composite index on Transactions(account_id, ts) together with join-helper indexes on AccountCustomer and Account. Lookup-oriented queries (Q1–Q3) and minimal computations (Q4–Q5) change little at this data size, with a few minor regressions that are typical when full scans are already inexpensive. Q11 (loan exposure by branch) improves by about 2× due to faster status filtering and branch resolution.

Table 6: Experiment 1 and 2 Speedup Analysis

Query	Baseline	Optimization	Speedup (x)	Rows returned
	Average Time (seconds)	Average Time (seconds)		
Q1	0.002649	0.008352	0.32	546
Q2	0.002356	0.006423	0.37	546
Q3	0.003223	0.004687	0.69	1000
Q4	0.000024	0.000026	0.92	1
Q5	0.000055	0.000074	0.74	27
Q6	0.21548	0.037727	5.71	100
Q7	0.392043	0.074861	5.24	110
Q8	0.51137	0.08457	6.05	110
Q9	0.592367	0.053515	11.07	1000



Q10	0.529291	0.045118	11.73	1000
Q11	0.000471	0.000229	2.06	5

Figure X below shows the speedups of Experiment 1 versus Experiment 2. The blue Bars show per-query speedup, sorted from highest to lowest, with the most significant gains on multi-branch dispersion, followed by grouping queries. The orange line represents the cumulative share of total improvement, indicating that these heavy queries account for nearly all of the performance benefit.

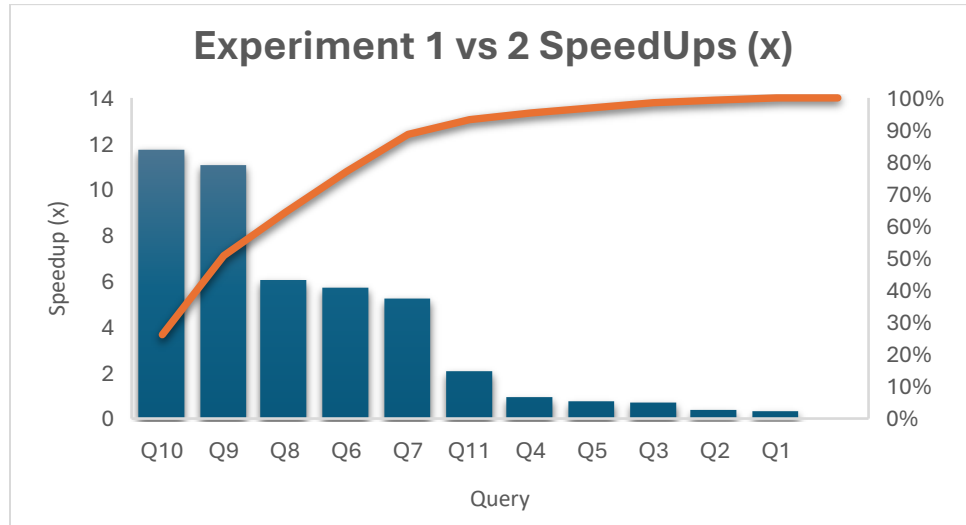


Figure X: Experiment 1 vs 2 Speedups

Lessons Learned

Several lessons were learned through this midterm report. Indexing strategy matters most for time-windowed and grouping workloads: a single composite index on Transactions(*account_id*, *ts*) plus join-helper indexes shifted heavy queries (Q6–Q10) from table scans to selective range scans, yielding 5–12× speedups, while simple lookups (Q1–Q3) saw little benefit at this scale. Accurate, labeled workloads and consistent run settings were essential for fair comparisons; anchoring filters to realistic time windows prevented “empty” results that can skew averages.

Resource Software Application

To maintain organization and track revisions, the entire project was managed through GitHub [10], which provides distributed version control and change tracking. Git commitments were used to document milestone progress (proposal, data generation, query tuning, and reporting). Collaboration across team members ensured that schema changes, experimental configurations, and scripts remained synchronized throughout development.

Microsoft Office 365, including OneDrive, Word, PowerPoint, and Excel, has been utilized by each team member to collaborate on progress and the final report. SQLite3 and Visual Studio Code are the primary tools for building, running, and testing the DB.



Milestone Achievements

Table X displays the milestone event schedule outlined in the project proposal. “Done” represents the completed milestone. “In-Progress” represented milestones that are currently ongoing. “Not Started” represents milestones that have not yet begun and are expected to be completed according to the schedule.

Table X: Milestone Project Tracker

Milestone # and Description	Status
Project Proposal	Done
Milestone #1: Generate data and Database	Done
Milestone #2: Queries	In-Progress
Midterm Progress Report	Done
Milestone #3: Scaling & Testing	In-Progress
Milestone #4: Validation & Metrics	In-Progress
Final Project Report	Not Started

Plan To Move Forward

1. Fine-tune the workload queries (tighten filters, standardize cutoffs, and add one or two diagnostics) and rerun across all experiments.
2. Upscale data (3×, 5×, 8×, 10×) and execute the full workload in both Baseline and Optimized modes for each scale, reporting speedups, index build time/size, and any regressions to validate robustness at larger volumes
3. Elaborate on the Analysis & Finalized Report

References

- [1] C. o. t. E. U. European Parliament, "General Data Protection Regulation (GDPR) - Legal Text," 14 April 2016. [Online]. Available: <https://gdpr-info.eu/>.
- [2] F. T. Commissions, "Federal Trade Commissions Protecting America's Consumers," 1999. [Online]. Available: <https://www.ftc.gov/business-guidance/privacy-security/gramm-leach-bliley-act>.
- [3] TPC, "TPC," [Online]. Available: <https://www.tpc.org/>.
- [4] H. U. J. D. & W. J. Garcia-Molina, Database Systems: The Complete Book, 3rd, Ed., Pearson, 2020.
- [5] R. & G. J. Ramakrishnan, Database Management Systems, 3rd, Ed., McGraw-Hill, 2003.
- [6] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th Edition ed., McGraw Hill Education.
- [7] ANSI, "The SQL Standard – ISO/IEC 9075:2023 (ANSI X3.135)," ANSI, [Online]. Available: <https://blog.ansi.org/ansi/sql-standard-iso-iec-9075-2023-ansi-x3-135/>.
- [8] P. S. Foundation, "Python Language," Python , 2025. [Online]. Available: <https://www.python.org/>.
- [9] S. Consortium, "SQLite," SQLite, 2025. [Online]. Available: <https://sqlite.org/>.



[10] GitHub, "ECE501C-BankingDB," GitHub, 2025. [Online]. Available: <https://github.com/sandraelenac/ECE501C-BankingDB>.